

Session 12 : Optimisation — Structures Spatiales, LOD et Billboards

💡 Objectifs de la session

Jusqu'ici, chaque session a ajouté de la **qualité** : éclairage, textures, ombres, post-processing. Aujourd'hui on s'attaque au problème inverse : **comment garder 60 FPS quand la scène grandit**. Une forêt avec 10 000 arbres, une ville avec 500 bâtiments, un champ d'herbe avec 50 000 brins — si on rend tout, le GPU s'effondre. Il y a deux leviers : **ne pas dessiner ce qu'on ne voit pas** (culling, structures spatiales) et **dessiner du faux pas cher** (LOD, billboards, impostors). C'est la session qui fait passer d'une démo à un jeu.

💡 Rappel : ce qu'on sait déjà

- **Session 2** : le scene graph organise les objets en hiérarchie parent-enfant. Le culling parcourt ce graphe pour décider quoi rendre.
- **Session 7** : une texture est une image appliquée sur une surface. Le billboard est une texture sur un quad — la "3D" est dans l'image, pas dans la géométrie.
- **Session 8** : les render targets permettent de rendre dans une texture. L'impostor utilise cette technique pour capturer un mesh dans une image.
- **Physique** : vous avez déjà vu les octrees et BVH pour le **broadphase collision** — on les retrouve ici, mais la question change : "est-ce visible ?" au lieu de "est-ce en collision ?".

Partie 1 : Le Problème des Grandes Scènes

💡 Le coût du rendu

Chaque objet rendu coûte :

- **CPU** : calculer les matrices, tester la visibilité, préparer les draw calls. C'est le **batch** — soumettre la géométrie au GPU.
- **GPU** : traiter les vertices (vertex shader), rasteriser, traiter les fragments (fragment shader), écrire dans le framebuffer.

Avec 10 000 objets, le CPU passe son temps à préparer des draw calls, et le GPU à traiter des fragments sur des objets invisibles. Les deux sont des goulots.

Les deux leviers

1. **Côté CPU — ne pas soumettre** : si un objet n'est pas dans le champ de la caméra, ne l'envoie même pas au GPU. C'est le **culling**.
2. **Côté GPU — dessiner moins cher** : si un objet est loin, il est petit à l'écran. Remplace sa géométrie par une version simplifiée (LOD) ou par une simple image (billboard / impostor).

FPS = 60 ← ne rien faire pour ce qu'on ne voit pas, et faire simple pour ce qu'on voit mal

Partie 2 : Culling — Ne Pas Dessiner l’Invisible

1. Frustum Culling

Le frustum

La caméra voit un volume en forme de pyramide tronquée — le **frustum** (Session 2). Tout ce qui est en dehors de ce volume n’apparaîtra jamais à l’écran. Le **frustum culling** élimine ces objets **avant** de les envoyer au GPU.

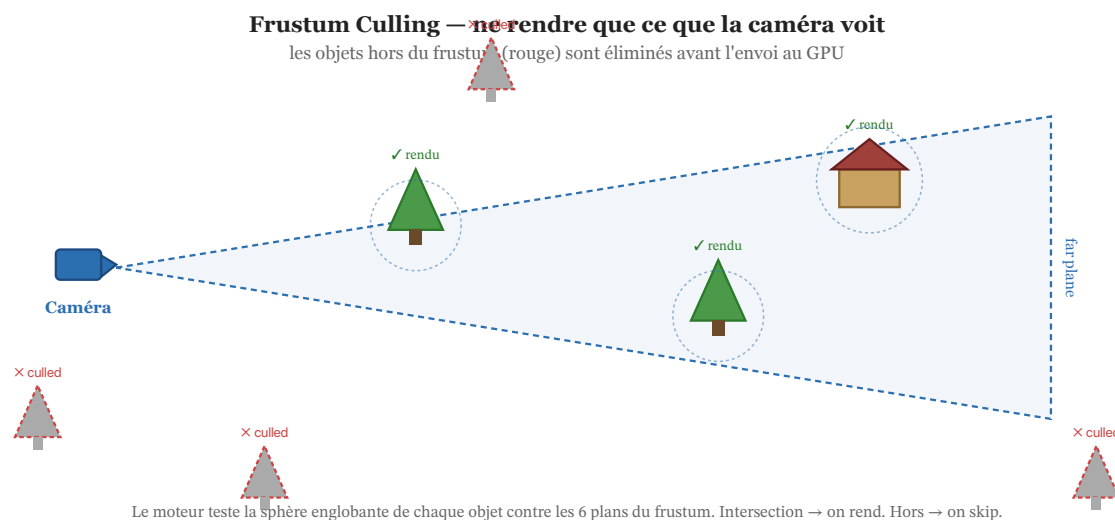


Figure 1: Frustum culling. Les objets verts (dans le frustum) sont rendus ; les objets rouges (hors frustum) sont éliminés sur le CPU. Le moteur teste la sphère englobante (bounding sphere) de chaque objet contre les 6 plans du frustum.

🔗 Bounding volumes

Pour tester rapidement si un objet est dans le frustum, on l’enveloppe dans une forme simple — **bounding sphere** (une sphère) ou **AABB** (Axis-Aligned Bounding Box). Tester une sphère contre 6 plans est 6 produits scalaires — quasi gratuit. Si la sphère est entièrement hors d’un plan, l’objet est culled.

2. Backface Culling

Déjà vu en Session 1

Le GPU élimine les triangles dont la normale pointe **away** de la caméra. C’est gratuit — c’est un paramètre du pipeline (`gl.enable(GL_CULL_FACE)`). On l’a vu avec l’ordre des indices : [0, 2, 1] inverse le winding et rend le triangle visible depuis l’autre côté.

3. Occlusion Culling

💡 Le plus difficile

Le frustum culling élimine ce qui est hors champ. Mais un objet **dans** le frustum peut être **caché** derrière un mur. Le **occlusion culling** détecte ces cas. C’est beaucoup plus dur :

- **Portals** : diviser la scène en pièces (portails). Si le portail est hors de vue, toute la pièce derrière est culled. Utilisé dans les jeux en intérieur.
- **GPU occlusion queries** : demander au GPU “est-ce que cet objet a des pixels visibles ?” avant de le rendre. Coûteux, souvent fait avec un délai.
- **Hi-Z buffer** : utiliser une version basse résolution du depth buffer pour tester l’occlusion sur le CPU.

La plupart des moteurs utilisent une combinaison. C’est un sujet avancé — on le mentionne, on ne l’implémente pas.

4. Structures Spatiales — Octree, BVH, Quadtree

Le problème du culling naïf

Sans structure, on teste chaque objet contre le frustum : $O(N)$ pour N objets. Avec 10 000 objets, c’est 10 000 tests par frame — sur le CPU, c’est un coût non négligeable. On peut faire mieux en **organisant** les objets dans l’espace.

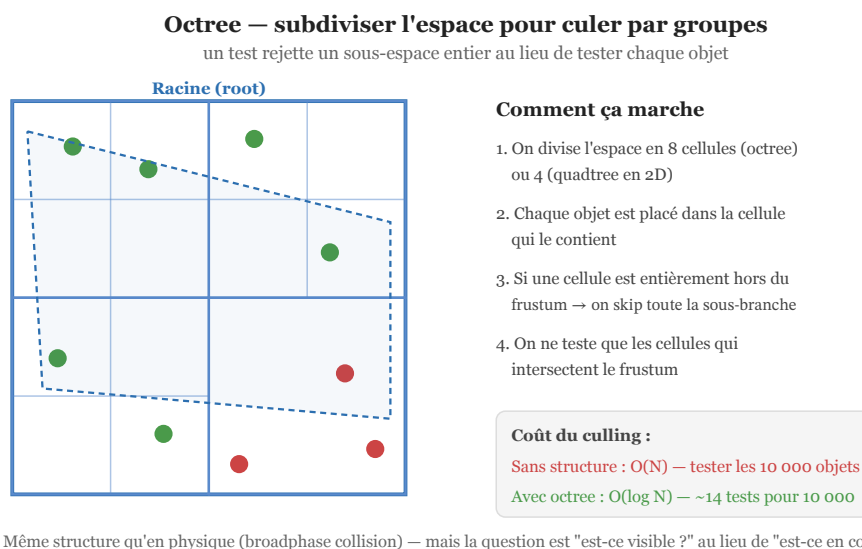


Figure 2: Octree : l’espace est divisé récursivement en 8 cellules (4 en 2D = quadtree). Chaque objet est placé dans la cellule qui le contient. Si une cellule entière est hors du frustum, on skip toute sa sous-branche — un seul test élimine des centaines d’objets.

Les structures courantes

- **Octree** : divise l’espace 3D en 8 cellules par niveau. Simple, adaptatif. Bon pour les scènes avec distribution variable.
- **BVH** (Bounding Volume Hierarchy) : arbre de bounding volumes (sphères ou AABB). Chaque nœud contient ses enfants. Bon pour les scènes avec des objets de tailles variées.
- **Quadtree** : version 2D de l’octree — 4 cellules par niveau. Utilisé pour les terrains.
- **Spatial hashing** : grille régulière avec hash. Très simple, excellent pour les objets uniformément distribués.
- **BSP** (Binary Space Partition) : division par plans. Historique (Quake), encore utilisé pour les niveaux en intérieur.

💡 Connexion avec le cours de physique

Vous avez déjà rencontré ces structures en physique pour le **broadphase collision** — l’octree ou le sweep-and-prune réduisent les $O(N^2)$ paires potentielles. En rendu, c’est la même structure, mais la requête est différente : “quels objets sont dans le frustum ?” au lieu de “quels objets sont proches les uns des autres ?”. La structure est identique, la fonction de requête change.

💬 En pratique : le moteur le fait déjà

Babylon.js et Three.js font le frustum culling automatiquement sur chaque mesh, en utilisant la bounding sphere. Pour les scènes très grandes, Three.js fournit BatchedMesh et Babylon.js fournit Thin Instances pour réduire les draw calls. Les structures spatiales custom (octree, BVH) sont rarement nécessaires dans un projet étudiant — mais comprendre **pourquoi** le moteur les utilise est essentiel pour diagnostiquer les problèmes de performance.

Partie 3 : LOD — Level of Detail

Le principe

Un objet lointain occupe peu de pixels à l’écran. Dessiner ses 17 000 triangles pour qu’ils couvrent 20 pixels est du gaspillage. Le **LOD** (Level of Detail) remplace la géométrie par une version simplifiée selon la distance à la caméra.

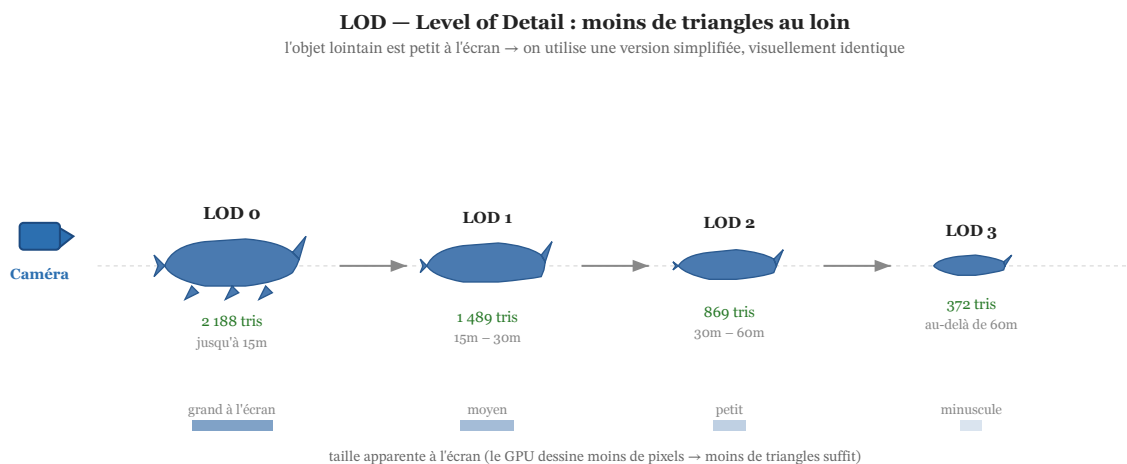


Figure 3: LOD sur un requin. LOD 0 (2 188 tris) près de la caméra, LOD 1 (1 489) à distance moyenne, LOD 2 (869) loin, LOD 3 (372) très loin. La taille apparente diminue, le nombre de triangles aussi — visuellement identique à l’écran.

1. Comment créer les niveaux de détail

Trois approches

- **Manuelle** : l’artiste modélise plusieurs versions. Qualité maximale, mais coûteux en temps de production.
- **Décimation** : un outil réduit automatiquement le nombre de triangles en fusionnant les vertices proches. Dans Blender, c’est le modifier **Decimate**. Rapide, qualité correcte.
- **Procedural** : générer les LOD à l’import (glTF peut contenir plusieurs meshes, ou le moteur les génère automatiquement). Godot et Unreal font ça automatiquement.

💡 Notre approche pour la démo

On a préparé un asset de requin avec 4 niveaux de LOD dans Blender, à partir d'un mesh retopologisé (Shark_low, 1 094 polys de base, 2 188 avec le modifier Mirror appliqué). En utilisant le modifier **Decimate** avec des ratios de 0.5, 0.25 et 0.1, on obtient 4 niveaux. Le tout est exporté dans un seul fichier `shark_lod.glb` qui contient les 4 meshes avec textures. Dans Three.js, on charge le GLB et on construit un objet `THREE.LOD` qui sélectionne le bon mesh selon la distance.

2. Quand passer d'un LOD à l'autre

💬 Le seuil de distance

Chaque LOD a une **distance de transition** — la distance à laquelle on passe au niveau suivant. Trop tôt : on voit le pop-in (changement brusque de géométrie). Trop tard : on gaspille des triangles.

Règle empirique : baser le seuil sur la **taille apparente à l'écran** (en pixels), pas sur la distance brute. Un objet de 1 m à 10 m et un objet de 100 m à 1000 m ont la même taille apparente.

Pop-in et comment l'atténuer

Le **pop-in** est le changement brusque visible quand on passe d'un LOD à l'autre. Solutions :

- **Seuils bien choisis** : assez loin pour que la différence soit imperceptible.
- **LOD cross-fade** : fondre entre deux LOD sur quelques frames (dither ou alpha blend). Three.js supporte ça via `lod.fadeType`.
- **Plus de niveaux** : 4-5 LODs réduisent le pop-in par rapport à 2.
- **Impostor au dernier niveau** : remplacer le LOD le plus lointain par un billboard (Partie 4).

3. LOD dans les moteurs

Support natif

- **Three.js** : `THREE.LOD` — on ajoute des meshes avec `addLevel(mesh, distance)`. Le moteur sélectionne automatiquement.
- **Babylon.js** : `detailMap` ou `meshLODLevel` avec `addLODLevel(distance, mesh)`.
- **Godot** : génère les LOD automatiquement à l'import du glTF.
- **Unreal / Unity** : systèmes de LOD intégrés, génération automatique avec paramètres de qualité.

Partie 4 : Billboards — la 3D dans une Texture

Le billboard

Un **billboard** est un quad (2 triangles) qui **fait toujours face à la caméra**. La géométrie est plate — la "3D" est dans la texture appliquée. Coût : 4 vertices, 2 triangles, 1 texture. C'est l'objet 3D le moins cher possible.

Billboard — un quad qui fait toujours face à la caméra

la géométrie est un simple rectangle ; la "3D" est dans la texture

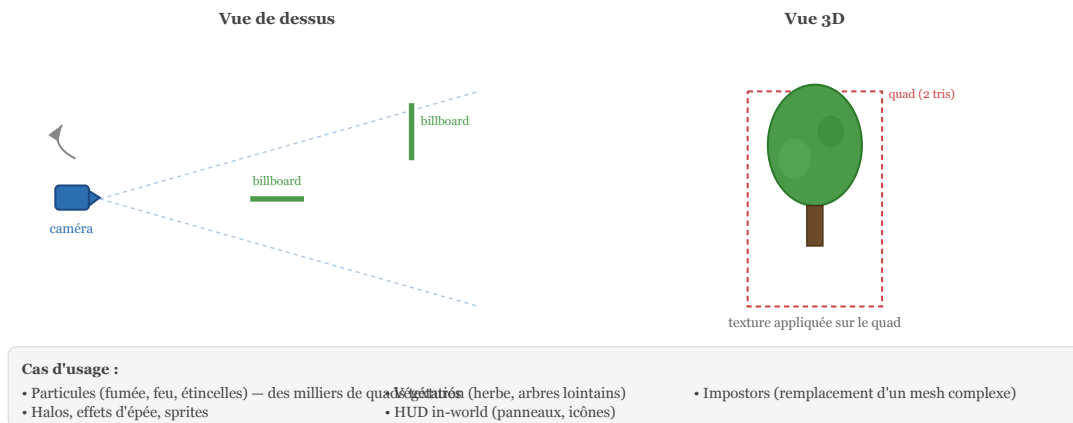


Figure 4: Un billboard. Vue de dessus (gauche) : le quad pivote pour rester perpendiculaire au rayon caméra. Vue 3D (droite) : la texture contient l'image de l'objet. Le résultat est un arbre qui "regarde" toujours la caméra — l'illusion tient tant qu'on ne tourne pas autour.

1. Modes de billboard

Spherical vs Cylindrical

- **Spherical** : le quad fait face à la caméra dans **tous les axes**. Utilisé pour les particules, les sprites, les halos. Problème : un arbre billboard spherical se "penche" quand on regarde vers le bas — il ne reste pas vertical.
- **Cylindrical** : le quad fait face à la caméra **autour de l'axe Y seulement**. Il reste vertical, ne fait que pivoter sur le plan horizontal. Utilisé pour les arbres, l'herbe, les objets posés au sol.

spherical : rotation(X, Y, Z) (caméra)

cylindrical : rotation(Y) (caméra) (vertical)

2. Cas d'usage

Où on les voit

- **Particules** : fumée, feu, étincelles, poussière. Des milliers de billboards animés. C'est 90% des effets visuels dans un jeu.
- **Végétation** : herbe sur un terrain (50 000 brins), arbres lointains. On ne modélise pas chaque brin — on pose des billboards.
- **Impostors** : un mesh complexe lointain remplacé par un billboard texturé avec une image du mesh (Partie 5).
- **Sprites et HUD in-world** : panneaux, icônes, marqueurs d'interaction qui doivent toujours être visibles.

3. Implémentation en Three.js

Billboard basique.

```
// Un plan avec une texture d'arbre
const texture = new THREE.TextureLoader().load('tree.png');
const material = new THREE.MeshBasicMaterial({
  map: texture,
  transparent: true,           // alpha pour les bords de l'arbre
```

```

    alphaTest: 0.5, // rejeter les pixels transparents (pas de tri de
profondeur)
    side: THREE.DoubleSide
});
const billboard = new THREE.Mesh(
    new THREE.PlaneGeometry(1, 1),
    material
);

// Faire face à la caméra (spherical) :
// Dans la boucle de rendu :
billboard.quaternion.copy(camera.quaternion);

// Ou en cylindrique (Y seulement) :
billboard.lookAt(camera.position.x, billboard.position.y, camera.position.z);

```

💡 Alpha test vs alpha blend

Les billboards utilisent des textures avec transparence (PNG avec alpha). Deux options :

- **Alpha blend** (transparent: true) : mélange la couleur avec le fond. Problème : nécessite un tri par distance (les objets transparents ne s'écrivent pas dans le depth buffer).
- **Alpha test** (alphaTest: 0.5) : rejette les pixels dont l'alpha est en dessous du seuil. Pas de mélange, écrit dans le depth buffer, pas besoin de tri. Préféré pour l'herbe et les feuilles — le bord est un peu dur, mais c'est gratuit.

4. Des milliers de billboards — Instancing

💡 Le problème des draw calls

Un billboard = 1 draw call. 50 000 brins d'herbe = 50 000 draw calls = catastrophe. Le CPU ne peut pas soumettre 50 000 commandes par frame.

Solution : **instancing**. On envoie la géométrie du quad **une seule fois** au GPU, et on fournit un tableau de **matrices de transformation** (une par instance). Le GPU duplique le quad en interne. 50 000 instances = 1 draw call.

Thin instances en Three.js.

```

const geometry = new THREE.PlaneGeometry(0.2, 0.5);
const material = new THREE.MeshBasicMaterial({
    map: grassTexture,
    alphaTest: 0.5,
    side: THREE.DoubleSide
});
const grass = new THREE.InstancedMesh(geometry, material, 50000);

// Positionner chaque brin d'herbe aléatoirement
const matrix = new THREE.Matrix4();
for (let i = 0; i < 50000; i++) {
    const x = (Math.random() - 0.5) * 100;
    const z = (Math.random() - 0.5) * 100;
    const y = 0;
    matrix.makeTranslation(x, y, z);
    grass.setMatrixAt(i, matrix);
}

```

```
grass.instanceMatrix.needsUpdate = true;
scene.add(grass); // 1 draw call pour les 50 000
```

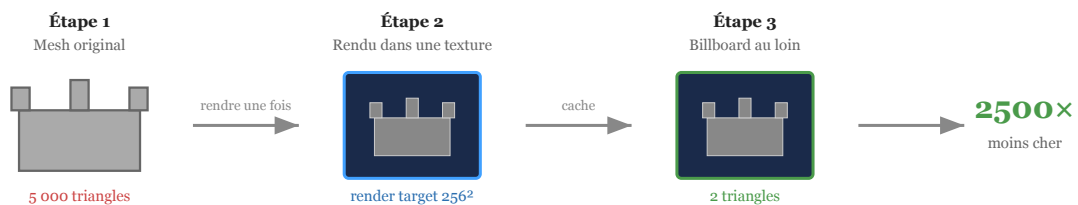
Partie 5 : Impostors — le LOD Extrême

L'impostor

Un **impostor** est la combinaison de LOD et billboard : on rend un mesh complexe **une fois** dans une texture (render target, Session 8), puis on affiche cette texture sur un billboard. Au lieu de dessiner 5 000 triangles chaque frame, on dessine 2 triangles — le quad avec l'image.

Impostor — le LOD extrême : un quad texturé au lieu d'un mesh

on rend le mesh une fois dans une texture, puis on affiche la texture sur un billboard



L'impostor est mis à jour périodiquement (ou par angle) quand l'objet se rapproche. Au-delà d'une certaine distance, il est visuellement identique au mesh.

Figure 5: Pipeline d'impostor. Le mesh original (5 000 tris) est rendu une fois dans une texture 256^2 . Au loin, on affiche la texture sur un billboard (2 tris). Résultat : 2 500× moins de triangles, visuellement identique à distance.

💡 Quand l'impostor se met à jour

L'impostor est une **vue figée** du mesh. Si la caméra tourne autour, l'image ne correspond plus. Solutions :

- **Impostor statique** : une seule vue, acceptable au très loin (l'objet est petit, l'angle change peu).
- **Impostor par angle** : pré-rendre plusieurs vues (8 ou 16 angles) et choisir la plus proche. Coût mémoire, mais qualité.
- **Mise à jour dynamique** : re-rendre l'impostor quand l'angle change trop. Coût GPU ponctuel, mais qualité maximale.

Dans la pratique, l'impostor est le dernier LOD avant de disparaître — au-delà d'une certaine distance, l'objet est trop petit pour qu'on voie la différence.

Partie 6 : Implémentation en Three.js

1. THREE.LOD — sélection automatique

Charger et utiliser les LOD du requin.

```
import { GLTFLoader } from 'three/addons/loaders/GLTFLoader.js';

const loader = new GLTFLoader();
loader.load('shark_lod.glb', (gltf) => {
  // Le GLB contient 4 meshes : LOD0, LOD1, LOD2, LOD3
  const lod = new THREE.LOD();
```

```
// Trouver chaque mesh par nom et l'ajouter avec sa distance
gltf.scene.traverse((child) => {
  if (child.isMesh) {
    if (child.name.includes('LOD0')) lod.addLevel(child, 0); // < 15m
    if (child.name.includes('LOD1')) lod.addLevel(child, 15); // 15-30m
    if (child.name.includes('LOD2')) lod.addLevel(child, 30); // 30-60m
    if (child.name.includes('LOD3')) lod.addLevel(child, 60); // > 60m
  }
});

scene.add(lod);
});

// Dans la boucle de rendu :
lod.update(camera); // Three.js sélectionne le bon mesh
```

2. Profiler — compter les draw calls

! Le réflexe à avoir

Avant d'optimiser, il faut **mesurer**. Three.js fournit `renderer.info` qui compte les draw calls, triangles, et textures :

```
console.log(renderer.info);
// { calls: 142, triangles: 285000, textures: 12, geometries: 8 }
```

Si `calls` est dans les milliers, c'est un problème de draw calls → instancing. Si `triangles` est dans les millions, c'est un problème de LOD → ajouter des niveaux.

3. La scène complète

Architecture typique

Une scène optimisée combine toutes les techniques :

1. **Frustum culling** (automatique) — élimine ce qui est hors champ.
2. **LOD** sur les objets complexes — moins de triangles au loin.
3. **Instancing** pour les objets répétés (herbe, arbres, rochers) — 1 draw call pour des milliers.
4. **Billboards** pour les particules et effets — les moins chers.
5. **Impostors** pour les objets très lointains — 2 triangles au lieu de milliers.

Partie 7 : Démonstration en Direct

! Démo interactive

On part d'une scène avec un terrain et on y ajoute :

1. Le requin avec 4 niveaux de LOD — on voit le triangle count changer dans `renderer.info` quand on zoome et dézoome.
2. Un champ d'herbe en instanced billboards (10 000 brins) — 1 draw call.
3. Un système de particules (billboards sphériques) — fumée ou étincelles.
4. Un compteur de FPS et de draw calls à l'écran.

On désactive le frustum culling et on voit les draw calls exploser. On active l'instancing et on voit les draw calls chuter.

Partie 8 : Pratique — Scène Optimisée

Cahier des charges. Construire une scène avec :

1. Le requin LOD (`shark_lod.glb`) — un `THREE.LOD` qui bascule entre les 4 niveaux.
2. Un terrain simple (plane subdivisée) avec au moins 1 000 brins d’herbe en instanced billboards.
3. Un système de particules (au choix : fumée, feu, étincelles) en billboards.
4. Un compteur de FPS et de draw calls (`renderer.info`) affiché à l’écran.
5. Un contrôle GUI pour ajuster les distances de transition LOD en temps réel.

Étapes — Three.js

1. Charger `shark_lod.glb` avec `GLTFLoader`, construire un `THREE.LOD` avec les 4 meshes.
2. Créer un terrain (plane 100×100, subdivisée).
3. Créer un `InstancedMesh` de quads texturés pour l’herbe, positionner aléatoirement.
4. Créer un système de particules (tableau de billboards ou `Points`).
5. Afficher `renderer.info.calls` et `renderer.info.triangles` dans un HUD.
6. Ajouter `lil-gui` pour ajuster les distances LOD.

Préparation de l’asset LOD dans Blender

💡 Comment on a préparé le requin

L’asset `shark_lod.glb` a été préparé dans Blender à partir d’un mesh retopologisé (`Shark_low`, 1 094 triangles de base) :

1. Le mesh original a un modifier **Mirror** (une moitié modelée, l’autre est le reflet). On applique le **Mirror** pour obtenir le mesh complet (2 188 triangles).
2. On duplique le mesh 3 fois → 4 copies.
3. Sur chaque copie, on ajoute un modifier **Decimate** avec un **ratio** de 0.5, 0.25 et 0.1.
4. On applique les modifiers.
5. On exporte en GLB avec les 4 meshes nommés `LOD0` à `LOD3`, avec la texture diffuse (2048²) et la normal map intégrées.

Le modifier **Decimate** en mode **Collapse** fusionne les vertices proches en préservant la silhouette. Le ratio n’est pas exact (1 489 au lieu de 1 094 pour 50%) car le mode **Collapse** préserve les bords et les contraintes topologiques. Le mode **Un-Subdivide** est une alternative qui supprime des boucles d’edges — meilleur pour les meshes avec une topologie régulière.

Critères de réussite

1. Le LOD bascule visiblement quand on s’éloigne (visible dans le compteur de triangles).
2. L’herbe s’affiche sans chute de FPS (1 draw call pour des milliers d’instances).
3. Le compteur de draw calls est visible et raisonnable (< 200).
4. Les distances LOD sont ajustables via la GUI.
5. Le pop-in est acceptable (pas de changement brusque évident à distance moyenne).

Conclusion

💬 Ce qu’on a vu

- **Culling** : frustum culling (hors champ), backface culling (dos des triangles), occlusion culling (caché derrière). Le moteur fait le frustum culling automatiquement.
- **Structures spatiales** : octree, BVH, quadtree organisent les objets pour culer par groupes. Même structure qu’en physique (broadphase), requête différente.

- **LOD** : remplacer la géométrie par une version simplifiée selon la distance. Créé par décimation dans Blender, géré par THREE.LOD dans Three.js.
- **Billboards** : un quad qui fait face à la caméra. La 3D est dans la texture. Particules, herbe, arbres lointains.
- **Instancing** : 1 draw call pour des milliers d'objets identiques. Indispensable pour l'herbe et les particules.
- **Impostors** : le LOD extrême — un mesh rendu dans une texture, affiché sur un billboard.

L'optimisation est ce qui sépare une démo d'un jeu. Une démo montre une technique ; un jeu en combine des dizaines, à 60 FPS, sur des scènes immenses. Les techniques d'aujourd'hui sont les fondations de toute scène de jeu réaliste.

💡 Récapitulatif du cours

En 12 sessions, on a parcouru tout le pipeline de rendu temps réel :

1. **Géométrie et transformations** (1-2) — comment on construit et place des objets 3D.
2. **Shaders** (3-4) — comment on contrôle le rendu au niveau du GPU.
3. **Éclairage** (5-6) — comment on simule la lumière, du direct au baked.
4. **Textures** (7-9) — comment on habille les surfaces, du simple au procédural.
5. **Ombres** (10) — comment on projette la lumière en temps réel.
6. **Post-processing** (11) — comment on traite l'image finale.
7. **Optimisation** (12) — comment on garde 60 FPS quand la scène grandit.

Vous avez maintenant les outils pour comprendre ce qui se passe dans n'importe quel moteur de jeu, et pour construire vos propres scènes optimisées.